

Cover

Securing User Accounts and File Permissions

Dennis, Nathan, Jason & Dominic

PNC Cohort

Supervision Under: Andry Rakotomalala

8/12/2026

Part 1: User Audit & Clean up

Part 1: User Audit & Clean up (User Account Lead)

- 1) List all local accounts (cat /etc/passwd) and identify any that are stale, duplicated, or no longer tied to an active team member.
 - 2) Check for accounts with UID 0 other than root, and for any accounts with no password set or with a blank/weak password.
 - 3) Remove or lock any account you determine shouldn't exist, documenting your reasoning for each.
 - 4) Confirm every remaining account has an appropriately restricted home directory (750 or stricter).
-

- 1) Run cat /etc/passwd

```
student@LinuxTargetVM4:~$ whoami
student
student@LinuxTargetVM4:~$ pwd
/home/student
student@LinuxTargetVM4:~$ cd /
student@LinuxTargetVM4:/$ pwd
/
student@LinuxTargetVM4:/$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:996:996:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
messagebus:x:101:101::/nonexistent:/usr/sbin/nologin
syslog:x:102:102::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:991:991:systemd Resolver:/:/usr/sbin/nologin
uidd:x:103:103::/run/uidd:/usr/sbin/nologin
tss:x:104:104:TPM software stack,,,:/var/lib/tpm:/bin/false
sshd:x:105:65534::/run/sshd:/usr/sbin/nologin
pollinate:x:106:1::/var/cache/pollinate:/bin/false
tcpdump:x:107:108::/nonexistent:/usr/sbin/nologin
landscape:x:108:109::/var/lib/landscape:/usr/sbin/nologin
fwupd-refresh:x:990:990:Firmware update daemon:/var/lib/fwupd:/usr/sbin/nologin
polkitd:x:989:989>User for polkitd:/:/usr/sbin/nologin
_chrony:x:109:113:Chrony daemon,,,:/var/lib/chrony:/usr/sbin/nologin
student:x:1000:1000:Ubuntu:/home/student:/bin/bash
oldservice:x:1001:1001::/home/oldservice:/bin/sh
postfix:x:110:115::/var/spool/postfix:/usr/sbin/nologin
_aide:x:988:988:Advanced Intrusion Detection Environment:/var/lib/aide:/usr/sbin/nologin
student@LinuxTargetVM4:/$
```

- 2) No accounts with unauthorized UID 0 privileges were identified; only root has UID 0. No accounts were found with blank password fields. Both student and oldservice have passwords configured. Existing password strength cannot be determined directly from the stored password hashes, so password complexity will be addressed through the PAM password policy in Part 2.

```
student@LinuxTargetVM4:/$ sudo awk -F: '$2 == "" {print $1}' /etc/shadow
student@LinuxTargetVM4:/$ sudo passwd -S student
student P 2026-08-05 0 99999 7 -1
student@LinuxTargetVM4:/$ sudo passwd -S oldservice
oldservice P 2026-08-05 0 99999 7 -1
student@LinuxTargetVM4:/$
```

- 3) The account 'oldservice' had an interactive /bin/sh shell and a configured password but was not associated with an active team member. After confirming it was not actively being used, the account was locked rather than deleted so its files and audit history would remain intact. Password status changed from P to L.

```
student@LinuxTargetVM4:~$ lastlog -u oldservice
Username      Port      From      Latest
oldservice
**Never logged in**
student@LinuxTargetVM4:~$ sudo change -l oldservice
sudo: change: command not found
student@LinuxTargetVM4:~$ sudo chage -l oldservice
Last password change           : Aug 05, 2026
Password expires                : never
Password inactive               : never
Account expires                 : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
student@LinuxTargetVM4:~$ ps -u oldservice
  PID TTY          TIME CMD
student@LinuxTargetVM4:~$ sudo passwd -S oldservice
oldservice P 2026-08-05 0 99999 7 -1
student@LinuxTargetVM4:~$ sudo passwd -l oldservice
passwd: password changed.
student@LinuxTargetVM4:~$ sudo passwd -S oldservice
oldservice L 2026-08-05 0 99999 7 -1
student@LinuxTargetVM4:~$
```

- 4) I used a command to check every local account that has a home directory under /home. The command pulled the username and home-directory path from /etc/passwd, then used stat to display the numeric permissions on each directory. I was checking to make sure every remaining user home directory was set to 750 or stricter, as required by the lab.

```
student@LinuxTargetVM4:/$ sudo ls -ld /home/*
drwxr-x--- 2 oldservice oldservice 4096 Aug  5 17:15 /home/oldservice
drwxr-x--- 5 student      student   4096 Aug 11 18:50 /home/student
student@LinuxTargetVM4:/$ awk -F: '$6 ~ /^\/home\/' {print $1, $6}' /etc/passwd | while read user home; do
[ -d "$home" ] && echo "$user $home $(stat -c '%a' "$home")"
done
student /home/student 750
oldservice /home/oldservice 750
frauduser /home/frauduser 750
loantest /home/loantest 750
student@LinuxTargetVM4:/$
```

*Run: `awk -F: '$6 ~ /^\/home\/' {print $1, $6}' /etc/passwd | while read user home; do
[-d "$home"] && echo "$user $home $(stat -c '%a' "$home")"
done*`

I audited the home-directory permissions for the remaining normal and test user accounts. Every user home directory was configured with permissions of 750 or stricter, so no permission changes were necessary. I excluded system/service directories because changing those could interfere with services or the operating system

```
student@LinuxTargetVM4:/$ while IFS=: read -r user _ uid _ _ home shell; do
if [ -d "$home" ]; then
printf "%-20s UID=%-6s HOME=%-30s PERMS=" "$user" "$uid" "$home"
stat -c '%a' "$home"
fi
done < /etc/passwd
```

root	UID=0	HOME=/root	PERMS=700
daemon	UID=1	HOME=/usr/sbin	PERMS=755
bin	UID=2	HOME=/bin	PERMS=777
sys	UID=3	HOME=/dev	PERMS=755
sync	UID=4	HOME=/bin	PERMS=777
games	UID=5	HOME=/usr/games	PERMS=755
man	UID=6	HOME=/var/cache/man	PERMS=755
mail	UID=8	HOME=/var/mail	PERMS=2775
proxy	UID=13	HOME=/bin	PERMS=777
backup	UID=34	HOME=/var/backups	PERMS=755
systemd-network	UID=998	HOME=/	PERMS=755
systemd-timesync	UID=996	HOME=/	PERMS=755
dhcpcd	UID=100	HOME=/usr/lib/dhcpcd	PERMS=755
systemd-resolve	UID=991	HOME=/	PERMS=755
uidd	UID=103	HOME=/run/uidd	PERMS=755
tss	UID=104	HOME=/var/lib/tpm	PERMS=755
sshd	UID=105	HOME=/run/sshd	PERMS=755
pollinate	UID=106	HOME=/var/cache/pollinate	PERMS=750
landscape	UID=108	HOME=/var/lib/landscape	PERMS=755
fwupd-refresh	UID=990	HOME=/var/lib/fwupd	PERMS=755
polkitd	UID=989	HOME=/	PERMS=755
_chrony	UID=109	HOME=/var/lib/chrony	PERMS=750
student	UID=1000	HOME=/home/student	PERMS=750
oldservice	UID=1001	HOME=/home/oldservice	PERMS=750
postfix	UID=110	HOME=/var/spool/postfix	PERMS=755
_aide	UID=988	HOME=/var/lib/aide	PERMS=700
frauduser	UID=1003	HOME=/home/frauduser	PERMS=750
loantest	UID=1004	HOME=/home/loantest	PERMS=750
fraudtest	UID=1005	HOME=/home/fraudtest	PERMS=750

```
student@LinuxTargetVM4:/$
```

Part 2 Password Policy

Part 2: Password Policy

- 1) Review and strengthen the existing password policy (minimum length, complexity, expiration) using PAM.

```
student@LinuxTargetVM4:~$ sudo nano /etc/security/pwquality.conf
student@LinuxTargetVM4:~$ sudo cat /etc/security/pwquality.conf
# Configuration for systemwide password quality limits
# Defaults:
#
# Number of characters in the new password that must not be present in the
# old password.
# difok = 1
#
# Minimum acceptable size for the new password (plus one if
# credits are not disabled which is the default). (See pam_cracklib manual.)
# Cannot be set to lower value than 6.
minlen = 12
#
# The maximum credit for having digits in the new password. If less than 0
# it is the minimum number of digits in the new password.
dcredit = -1
#
# The maximum credit for having uppercase characters in the new password.
# If less than 0 it is the minimum number of uppercase characters in the new
# password.
ucredit = -1
#
# The maximum credit for having lowercase characters in the new password.
# If less than 0 it is the minimum number of lowercase characters in the new
# password.
lcredit = -1
#
# The maximum credit for having other characters in the new password.
# If less than 0 it is the minimum number of other characters in the new
# password.
ocredit = -1
#
# The minimum number of required classes of characters for the new
# password (digits, uppercase, lowercase, others).
minclass = 4
#
# The maximum number of allowed consecutive same characters in the new password.
# The check is disabled if the value is 0.
maxrepeat = 3
```

(minlen = 12, dcredit = -1, ucredit = -1, lcredit = -1, ocredit = -1, minclass = 4, maxrepeat = 3)

```
student@LinuxTargetVM4:~$ sudo cat /etc/pam.d/common-password
#
# /etc/pam.d/common-password - password-related modules common to all services
#
# This file is included from other service-specific PAM config files,
# and should contain a list of modules that define the services to be
# used to change user passwords. The default is pam_unix.

# Explanation of pam_unix options:
# The "yescrypt" option enables
# hashed passwords using the yescrypt algorithm, introduced in Debian
# 11. Without this option, the default is Unix crypt. Prior releases
# used the option "sha512"; if a shadow password hash will be shared
# between Debian 11 and older releases replace "yescrypt" with "sha512"
# for compatibility. The "obscure" option replaces the old
# 'OBSOLETE_CHECKS_ENAB' option in login.defs. See the pam_unix manpage
# for other options.

# As of pam 1.0.1-6, this file is managed by pam-auth-update by default.
# To take advantage of this, it is recommended that you configure any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules. See
# pam-auth-update(8) for details.

# here are the per-package modules (the "Primary" block)
password      requisite          pam_pwquality.so retry=3
password      [success=1 default=ignore] pam_unix.so obscure use_authtok try_first_pass yescrypt
# here's the fallback if no module succeeds
password      requisite          pam_deny.so
# prime the stack with a positive return value if there isn't one already;
# this avoids us returning an error just because nothing sets a success code
# since the modules above will each just jump around
password      required           pam_permit.so
password      required           pam_pwhistory.so remember=999999 use_authtok
# and here are more per-package modules (the "Additional" block)
# end of pam-auth-update config
```

(The line “password requisite pam_pwquality.so retry=3” ensures that the pwquality file is called by PAM

The line “password required pam_pwhistory.so remember=999999 use_authtok” ensures that passwords cannot be reused)

```
student@LinuxTargetVM4:~$ sudo nano /etc/login.defs
```

```
PASS_MAX_DAYS 90
PASS_MIN_DAYS 0
PASS_WARN_AGE 14
```

(Password must be changed after 90 days, no minimum password age requirement, users get a password expiration warning 14 days before change is required)

```
student@LinuxTargetVM4:~$ cat /etc/security/faillock.conf
```

```
deny = 5
```

```
unlock_time = 900
```

(Faillock in place from last lab, login attempts limited to 5 before temp ban for 15 minutes)

```
student@LinuxTargetVM4:~$ sudo cat /etc/pam.d/common-auth
#
# /etc/pam.d/common-auth - authentication settings common to all services
#
# This file is included from other service-specific PAM config files,
# and should contain a list of the authentication modules that define
# the central authentication scheme for use on the system
# (e.g., /etc/shadow, LDAP, Kerberos, etc.). The default is to use the
# traditional Unix authentication mechanisms.
#
# As of pam 1.0.1-6, this file is managed by pam-auth-update by default.
# To take advantage of this, it is recommended that you configure any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules. See
# pam-auth-update(8) for details.
auth    required                                pam_faillock.so preauth
# here are the per-package modules (the "Primary" block)
auth    [success=1 default=bad]                pam_unix.so nullok
auth    [default=die]                          pam_faillock.so authfail
auth    sufficient                            pam_faillock.so authsucc
# here's the fallback if no module succeeds
auth    requisite                            pam_deny.so
# prime the stack with a positive return value if there isn't one already;
# this avoids us returning an error just because nothing sets a success code
# since the modules above will each just jump around
auth    required                            pam_permit.so
# and here are more per-package modules (the "Additional" block)
auth    optional                            pam_cap.so
# end of pam-auth-update config
```

(common-auth file calls upon the pam_faillock.so file enforcing the login lockout)

Part 3: File Permission & Ownership Audit

Part 3: File Permissions and Ownership Audit (File Permissions Lead)

- 1) Search for SUID and SGID binaries on the host: `find / -perm -4000 -o -perm -2000 -type f 2>/dev/null`, and determine whether each one is legitimately required.

```
student@LinuxTargetVM4:~$ find / -perm -4000 -o -perm -2000 -type f 2>/dev/null
/usr/bin/chage
/usr/bin/sudo
/usr/bin/mount
/usr/bin/gpasswd
/usr/bin/expiry
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/crontab
/usr/bin/umount
/usr/bin/ssh-agent
/usr/bin/newgrp
/usr/bin/fusermount3
/usr/bin/dotlockfile
/usr/bin/chfn
/usr/bin/su
/usr/sbin/postdrop
/usr/sbin/postqueue
/usr/sbin/mount.cifs
/usr/sbin/unix_chkpwd
/usr/sbin/pam_extrausers_chkpwd
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/x86_64-linux-gnu/utempter/utempter
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/openssh/ssh-keysign
```

- Audit confirmed that there were only legitimate system files with SUID and SGID binaries on the host

- 2) Search for world-writable files and directories, and correct any that shouldn't be writable by everyone.

```
student@LinuxTargetVM4:~$ sudo find / \( -type f -o -type d \) -perm -0002 -not -path "/proc/*" -not -path "/sys/*" -not
-path "/run/*" 2>/dev/null
/var/crash
/var/tmp
/var/tmp/systemd-private-588854b5bda942a29a4ee7ddf9ec75ca-chrony.service-FNIqYh/tmp
/var/tmp/systemd-private-588854b5bda942a29a4ee7ddf9ec75ca-fwupd.service-WNnC8g/tmp
/var/tmp/cloud-init
/var/tmp/systemd-private-588854b5bda942a29a4ee7ddf9ec75ca-systemd-resolved.service-v2swWM/tmp
/var/tmp/systemd-private-588854b5bda942a29a4ee7ddf9ec75ca-polkit.service-LM5bNr/tmp
/var/tmp/systemd-private-588854b5bda942a29a4ee7ddf9ec75ca-systemd-logind.service-lZ34GB/tmp
/tmp
/tmp/systemd-private-588854b5bda942a29a4ee7ddf9ec75ca-chrony.service-ZZZYp/tmp
/tmp/systemd-private-588854b5bda942a29a4ee7ddf9ec75ca-systemd-logind.service-DyGxdT/tmp
/tmp/systemd-private-588854b5bda942a29a4ee7ddf9ec75ca-systemd-resolved.service-v4ID2i/tmp
/tmp/.X11-unix
/tmp/systemd-private-588854b5bda942a29a4ee7ddf9ec75ca-polkit.service-HqB4kL/tmp
/tmp/systemd-private-588854b5bda942a29a4ee7ddf9ec75ca-fwupd.service-byIfiD/tmp
/tmp/.font-unix
/tmp/.XIM-unix
/tmp/.ICE-unix
/dev/mqueue
/dev/shm
student@LinuxTargetVM4:~$
```

- Search returned standard /tmp/ directories where any user or application on the system needs to be able to write temporary files so no remediation needed

- 3) Audit ownership of key application and configuration directories, correcting any files owned by the wrong user or group.

```
student@LinuxTargetVM4:~$ sudo find /etc \! -user root -type f 2>/dev/null | xargs ls -l
/etc/old_report.log
```

- Old_report.log is found in /etc/ directory where the owner is not root.

- 4) Document every permission or ownership change with before-and-after values.

Before:

User and Group owners set to old service

```
student@LinuxTargetVM4:~$ pwd
/home/student
student@LinuxTargetVM4:~$ ls -l /etc/old_report.log
-rw-r----- 1 oldservice oldservice 0 Aug  7 13:47 /etc/old_report.log
```

After:

User and Group owners changed to root

```
student@LinuxTargetVM4:~$ sudo chown root:root /etc/old_report.log
student@LinuxTargetVM4:~$ ls -l /etc/old_report.log
-rw-r----- 1 root root 0 Aug  7 13:47 /etc/old_report.log
```

Part 4: Access Control List for shared directory

Part 4: Access Control Lists for a Shared Directory (ACL Lead)

- 1) Create a shared directory (e.g., /srv/shared-cases) that both a loanops group and a fraud investigations group need access to, but with different permissions for each.

```
student@LinuxTargetVM4:~$ sudo mkdir -p /srv/shared-cases
student@LinuxTargetVM4:~$ ls -ld /srv/shared-cases
drwxr-xr-x 2 root root 4096 Aug 12 19:00 /srv/shared-cases
```

- Created /srv/shared-cases for both teams to access.

- 2) Create the two groups and assign appropriate team-created test accounts to each.

```
student@LinuxTargetVM4:/srv/shared-cases$ sudo groupadd loanops
student@LinuxTargetVM4:/srv/shared-cases$ sudo groupadd fraudinvestigations
student@LinuxTargetVM4:/srv/shared-cases$ sudo useradd -m -s /bin/bash loantest
student@LinuxTargetVM4:/srv/shared-cases$ sudo useradd -m -s /bin/bash fraudtest
student@LinuxTargetVM4:/srv/shared-cases$ sudo usermod -aG loanops loantest
student@LinuxTargetVM4:/srv/shared-cases$ sudo usermod -aG fraudinvestigations fr
audtest
student@LinuxTargetVM4:/srv/shared-cases$ id loantest
uid=1006(loantest) gid=1009(loantest) groups=1009(loantest),1007(loanops)
student@LinuxTargetVM4:/srv/shared-cases$ id fraudtest
uid=1007(fraudtest) gid=1010(fraudtest) groups=1010(fraudtest),1008(fraudinvestig
ations)
student@LinuxTargetVM4:/srv/shared-cases$
```

- Created the loanops group.
- Created the fraudinvestigations group.
- Created loantest and fraudtest as test accounts.
- Added loantest to loanops.
- Added fraudtest to fraudinvestigations.

- 3) Use setfacl to grant loanops read/write access and fraud investigations read-only access to the shared directory, without loosening permissions for anyone else.

```
student@LinuxTargetVM4:/srv/shared-cases$ sudo setfacl -m g:loanops:rwx .
student@LinuxTargetVM4:/srv/shared-cases$ sudo setfacl -m g:fraudinvestigations:rx .
student@LinuxTargetVM4:/srv/shared-cases$
```

- Used setfacl to give loanops read/write access.
- Used setfacl to give fraudinvestigations read-only access.

- 4) Verify the ACLs with getfacl and test access as each test account to confirm the permissions behave as intended

```
student@LinuxTargetVM4:/srv/shared-cases$ getfacl .
# file: .
# owner: root    20.2 MiB   0%    1%
# group: root    18.4 MiB   0%    1%
user::rwx
group::r-x 8.3 MiB   0%    0%
group:1002:rwx 6.5 MiB   0%    0%
group:1003:r-x
group:loanops:rwx 3 MiB   0%    0%
group:fraudinvestigations:r-x
mask::rwx
other::r-x 8.3 MiB   0%    0%

student@LinuxTargetVM4:/srv/shared-cases$ sudo -u loantest ls .
loantest 8.3 MiB   0%    0%
student@LinuxTargetVM4:/srv/shared-cases$ sudo -u loantest sh -c 'echo "Loan test works" > loantest-test'
student@LinuxTargetVM4:/srv/shared-cases$ cat loantest-test
Loan test works
```

- Used sudo -u loantest to test access without logging into the account.
- Successfully created/wrote to a test file.
- This confirmed that loanops has write access.

```
student@LinuxTargetVM4:/srv/shared-cases$ sudo -u fraudtest touch fraudtest-test
touch: cannot touch 'fraudtest-test': Permission denied
student@LinuxTargetVM4:/srv/shared-cases$
```

- Attempted to create a file as fraudtest.
- Received Permission denied, confirming that the group does not have write access.

Reflection Questions

Reflection Questions

1) What was the hardest part of dividing this work across the team, and how did you keep everyone's changes from conflicting on the shared host?

The hardest part of dividing the work was coordinating our efforts to not lock out any other team member or override any of each other configurations. We avoided this by clearly defining the scope of our lab and completing the lab parts with active communication within our team. For Part 1, it was clearly communicated among us that the 'old service' account was to be locked and recorded for lab documentation and then unlocked to continue with the lab as a precaution to prevent interfering with any other member's configurations.

2) Why do ACLs solve the shared-directory problem better than standard Linux owner/group/other permissions alone?

Standard Linux permissions only let you assign access to one owner and one group, so several different people or teams cannot access the same shared folder. ACLs (Access Control Lists) fix this by letting you assign specific permissions to multiple individual users or groups on the same file or directory, all at once. This flexibility is what shared directories with mixed access needs require, which basic owner/group/other permissions simply can't express.

3) How does adding MFA change the risk calculus for an account whose password is eventually guessed or leaked?

Adding MFA greatly reduces the risk if a password is guessed or leaked because the password alone is no longer enough to access the account. The attacker would also need the second factor, such as the one-time authentication code. This gives the organization another layer of protection and more time to detect the compromise and reset the exposed password.